Prof. MVIBUNDULU Gaëtan-069103715/053221067

CAHIER DES CHARGES LICENCE 3 - GI

Projet Web « Brigi+ Consulting »

Architecture MVT, Templates Globaux, URLs Nommées & Statiques (Sans BDD)

1. Contexte et Mission du Projet

L'objectif de ce projet à réaliser à la maison est de concevoir un site web vitrine interactif et complet pour une agence de voyage fictive nommée **Brigi+ Consulting**. Le site permettra de présenter l'agence, d'explorer le catalogue des offres, d'afficher la fiche détaillée d'une destination et de simuler la prise de contact.

> **CONTRAINTE MAJEURE :** Ce travail doit être réalisé exclusivement avec le framework Django en respectant l'architecture MVT. ATTENTION : Aucune base de données (modèles Django / ORM) ne doit être utilisée. L'objectif prioritaire est de maîtriser la structure des dossiers, le routage dynamique, l'héritage de templates et les fichiers statiques.

2. Spécifications Fonctionnelles (Pages du Site)

Le site web doit être composé au minimum de 5 pages principales partageant une mise en page unifiée (en-tête avec barre de navigation et pied de page réutilisables).

Page	URL visée	Description et contenu attendu
Accueil	/	Présentation générale de l'agence, slogan accrocheur et mise en valeur visuelle d'une sélection de voyages « coup de cœur ».
Catalogue Destinations	/destinations/	Liste complète de toutes les offres de voyage proposées par l'agence sous forme de cartes/vignettes.
Détail Destination	/destinations/<identifiant>/	Page dynamique affichant les détails complets d'un voyage sélectionné (prix, durée, pays, description complète).
À propos	/a-propos/	Présentation de l'agence, de ses valeurs, de son histoire et de son équipe.
Contact	/contact/	Formulaire de contact et de réservation statique permettant de simuler l'envoi d'une demande par un client.

3. Exigences Techniques et Travail d'Analyse

A. Gestion des Données Simulées (Mock Data)

- Créer un fichier Python dédié à la structure des données au sein de l'application (ex: **donnees.py**).
- Ce fichier doit contenir une collection (ex: liste de dictionnaires) répertoriant au minimum **4 à 5 destinations** distinctes.
- Chaque voyage doit comporter les caractéristiques suivantes : identifiant unique, titre, pays, prix, durée, chemin d'accès à l'image, description complète et statut coup de cœur (booléen).

B. Emplacement des Templates et Fichiers Statiques

- Le dossier des templates (**templates/**) doit être configuré obligatoirement à la **racine du projet** (et non à l'intérieur de l'application). La configuration appropriée doit être appliquée dans le fichier de paramètres.
- Mettre en place un template squelette (ex: **base.html**) contenant la structure HTML pro générale, la barre de navigation et le footer. Tous les autres templates doivent hériter de ce fichier parent.
- Tous les éléments graphiques (images des destinations) et le fichier de style CSS doivent être chargés via le mécanisme officiel des fichiers statiques de Django.

C. Routage et URLs Nommées

- Toutes les routes d'URL doivent obligatoirement être identifiées par un **nom explicite** (attribut **name**).
- Dans l'ensemble des fichiers HTML, **il est strictement interdit d'écrire des liens d'URL en dur** (ex: href="/destinations/"). Tous les liens (barre de navigation, cartes, boutons) doivent faire appel à la balise de template dédiée aux URLs nommées de Django.
- La route affichant le détail d'une destination doit capturer un paramètre dynamique (l'identifiant du voyage) afin de transmettre les informations correctes à la vue.

4. Livrables et Critères d'Évaluation

Chaque étudiant doit rendre l'archive du projet complet + le lien de votre application.

- **Organisation du projet :** Respect de l'arborescence demandée (templates à la racine, structuration des fichiers statiques, propreté du dossier).
- **Configuration Django :** Exactitude du paramétrage du dossier global de templates et des statiques dans les réglages du projet.
- **Exploitation du routage :** Utilisation conforme et stricte des URLs nommées dans tous les templates sans aucun lien codé en dur.
- **Logique des vues & contextes :** Capacité à filtrer et passer les données simulées depuis la vue vers les templates HTML.
- **Gestion du cas limite :** Mise en place d'une réaction appropriée (page ou erreur 404) si l'utilisateur saisit l'identifiant d'un voyage inexistant dans l'URL.